

Designing Good Backend Solutions



```
{ "status": "optimized" }
```

TA Ahmed Mohamed • TA Ahmed Mostafa



| What Is a Good Backend Solution?



Correct, fast, and reliable responses

The system delivers accurate data quickly, every time.



Scales as users grow

Handles increased traffic without crashing or slowing down.



Simple to understand and maintain

Code is clean and easy for other developers to work on.



Safe and cost-aware

Protects user data and optimizes resource usage.



Core Principles



Scalability

Handle more load by adding resources (vertical or horizontal scaling) without redesigning.



Reliability

Keep working correctly even when parts fail (hardware, network, or dependent services).



Simplicity

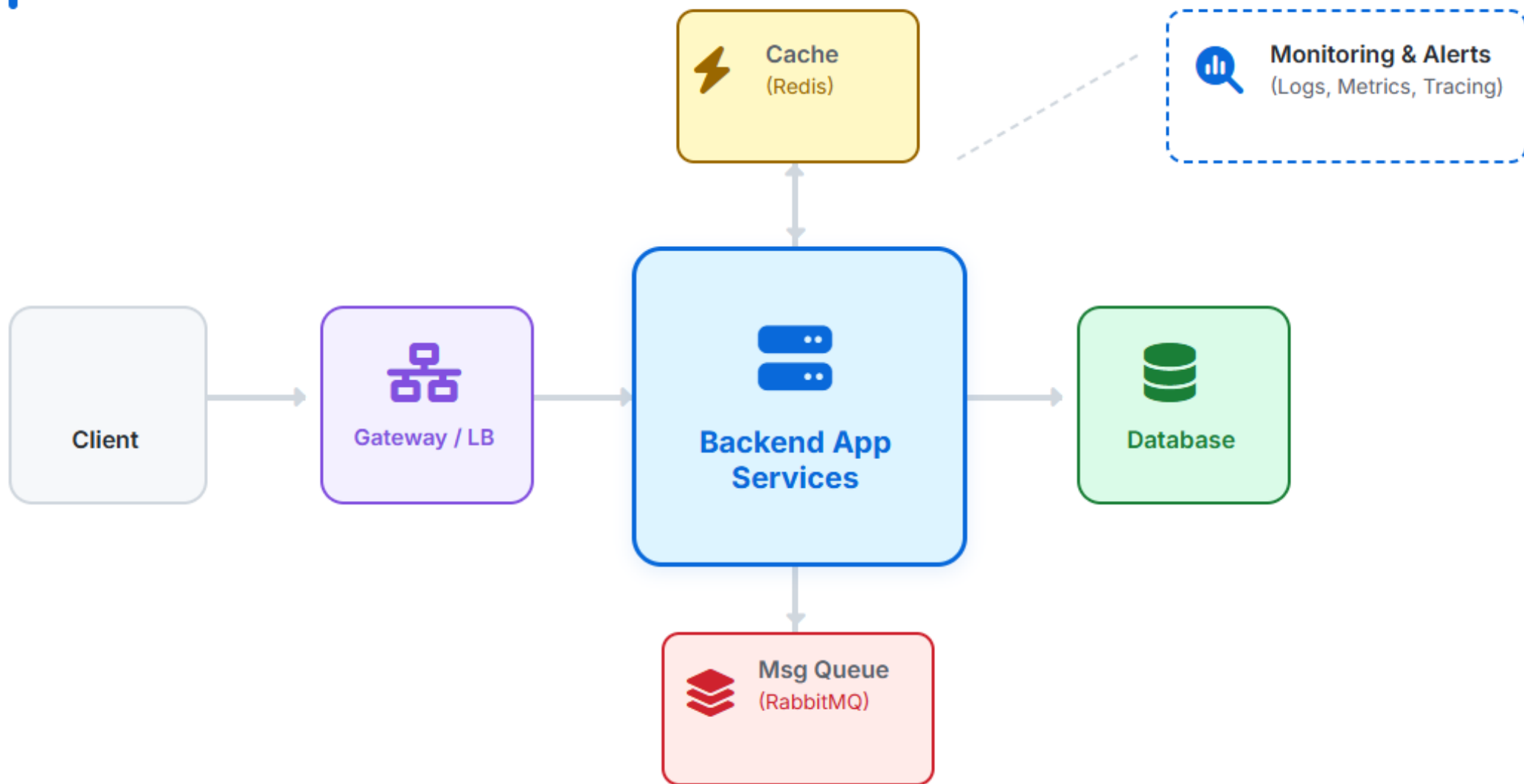
Fewer moving parts and clear code.
Complexity is the enemy of stability.



Maintainability

Easy to change, test, and deploy. New team members can understand it quickly.

Backend Architecture Basics



API Design Best Practices



Resource-based URLs and proper HTTP verbs

Use nouns (e.g., /users) and verbs (GET, POST).



Versioning (e.g., /v1)

Allows you to change the API without breaking clients.



Meaningful status codes + error bodies

Return 200, 201, 400, 404, 500 appropriately.



Idempotency for unsafe operations

Ensure retrying a request (like payment) doesn't duplicate it.



Validation and security (authn/z)

Always validate input; secure endpoints with JWT/OAuth.



Good docs (OpenAPI/Swagger)

Clear documentation helps consumers integrate easily.



Database Design Basics



Choose the right type

Decide between SQL (relational) vs NoSQL based on data structure.



Model entities and relationships

Define tables, keys, and how data connects clearly.



Index the hot queries

Speed up frequent lookups to prevent slow performance.



Use transactions where needed

Ensure data integrity for operations that must succeed or fail together.



Plan migrations and backups

Prepare strategies for schema changes and disaster recovery.



Caching Overview



What is Caching?

Store hot data temporarily to respond much faster.



Where to Cache

CDN (User edge), App Memory (Redis), or DB Buffer.



Keys, TTL, & Eviction

Use expiration times (TTL) and LRU policies to manage space.



Beware Stale Data

Old data can cause bugs; define clear invalidation rules.



| Logging & Monitoring



Structured logs with levels

Use JSON format and appropriate levels (INFO, WARN, ERROR) for easier parsing.



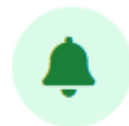
Tracing across services

Implement distributed tracing (e.g., OpenTelemetry) to follow requests through microservices.



Metrics (latency, error rate, RPS)

Track key performance indicators to understand system health in real-time.



Dashboards + alerts (SLIs/SLOs)

Visualize data and set alerts for when Service Level Objectives are breached.



| Error Handling & Resilience



Timeouts and retries (with backoff)

Stop waiting indefinitely; retry failures with exponential delays.



Fallbacks and graceful degradation

Serve cached or default data if the main source fails.



Rate Limiting: The Concept



Control Request Volume

Control exactly how many requests a client can make within a specific timeframe.



Protection & Fairness

Protects your service from overload and ensures fair resource usage for everyone.



Real-World Example

Allowing a maximum of 100 requests per minute per user or IP address.



Rate Limiting: Common Algorithms



Fixed Window

Counts requests in fixed time blocks (e.g., 10:00-10:01). Simple to implement, but allows double load at window edges.



Sliding Window

Tracks timestamps to smooth out the window. Prevents boundary spikes but requires more storage or processing.



Token Bucket

Tokens refill at a fixed rate. Each request uses a token. Allows short traffic bursts as long as tokens are available.



Leaky Bucket

Requests enter a queue and leave at a constant rate. Smooths out all bursts into a steady, predictable stream.

Rate Limiting: Use Cases & Trade-offs

✓ Use Cases



Public APIs

Ensure fair usage among all consumers.



Login Endpoints

Prevent brute-force password guessing.



Expensive Operations

Protect heavy reports or image processing.



Prevent Abuse

Stop scraping and DoS attacks.

⚖ Trade-offs



Added State & Storage

Need to store counters (e.g., Redis) which adds cost.



False Positives

Might block legitimate bursts of traffic from users.



Distributed Complexity

Synchronizing limits across multiple servers/regions is hard.

| Pagination: Concept & Why



Split large results into pages

Break down massive datasets into manageable chunks.



Faster responses and smaller payloads

Significantly reduces network latency and bandwidth usage.



Better UX and lower DB/memory cost

Prevents UI freezes and server-side memory exhaustion.



Pagination: Offset vs Cursor

Offset / Limit

```
GET /items?offset=100&limit=20
```

Traditional method. Skips N records and takes M.

- ✓ Simple to implement
- ✓ Can jump to specific page
- ✗ **Slow with large data**
Database must count & skip rows.
- ✗ **Unstable results**
New items shift the offset window.

Cursor / Keyset

```
GET /items?after=id_842&limit=20
```

Modern method. Points to the last item seen.

- ✓ Extremely fast ($O(1)$)
- ✓ Stable results with changes
- ✗ **Complex to implement**
Needs unique, sortable column (index).
- ✗ **Cannot jump to page**
Must traverse sequentially.

| Pagination: Best Practices



Always order by an indexed unique column

Ensures stable results and prevents items from appearing on multiple pages or being skipped.



Return next/prev cursors in response

Include opaque cursor strings in metadata so clients can navigate easily without calculating offsets.



Set reasonable max limits (e.g., 100)

Prevent database overload by capping the number of items returned in a single request.



Keep responses consistent

Ensure the data schema and sort order remain identical across all pages for a smooth experience.



| Design Patterns: Why Use Them?



Creational

How we create objects to avoid direct instantiation.

EXAMPLES

Singleton
Factory



Behavioral

How objects communicate and interact efficiently.

EXAMPLES

Strategy
Observer



Data Access

How we abstract and talk to data sources cleanly.

EXAMPLES

Repository

| Singleton & Factory in Backend

1

Singleton

Ensures a class has only one instance and provides a global point of access to it.

- ✓ **Shared Resources**
Database connection pools, logging services.
- ✓ **Avoid Global State Issues**
Ensure thread safety when accessing the instance.



Factory

Defines an interface for creating objects, but lets subclasses decide which class to instantiate.

- ✓ **Centralize Creation**
Manage complex logic for creating HTTP clients or parsers.
- ✓ **Flexibility**
Swap implementations (e.g., Mock vs Real) easily.

Strategy & Repository in Backend



Strategy

Enables selecting an algorithm's behavior at runtime.

- ✓ **Pluggable Algorithms**
Sorting, payment processing, or auth providers.
- ✓ **Runtime Flexibility**
Choose via interface without modifying clients.

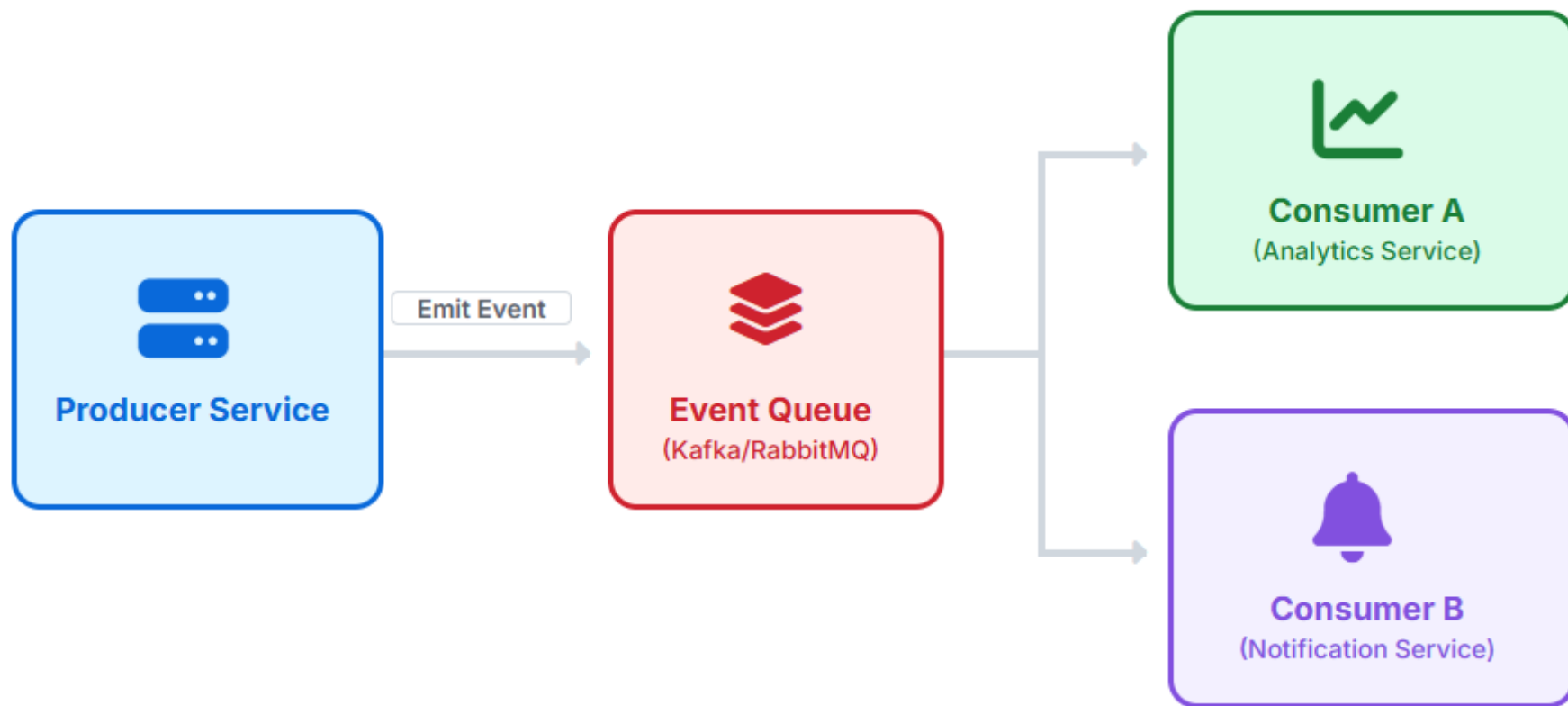


Repository

Encapsulates the logic required to access data sources.

- ✓ **Abstract Data Access**
Centralize CRUD operations; hide SQL/NoSQL details.
- ✓ **Decoupling**
Swap databases without changing business logic.

Observer / Event-Driven Pattern



Decouple producers and consumers



Scale consumers independently



Useful for **notifications, analytics, and ETL** jobs

Summary & Key Takeaways



Keep it simple,
observable, and resilient



Use caching, pagination,
and rate limits wisely



Choose patterns that
clarify your design

Thank You!

TA Ahmed Mohamed • TA Ahmed Mostafa